Producto 04 — Diseño del software

Servicio Nacional de Aprendizaje (SENA)
Programa: Análisis y Desarrollo de Software
Ficha: 3145555
Proyecto: SmartHome
Elaborado por: Natalia Chala Chala, Kevin Stiven López Amaya y Karen Daniela Holguín Cruz
Instructor: José de Jesús Motta Vargas
Fecha de actualización: 6 de noviembre de 2026

1. Presentación

Este documento actualiza el diseño del software de SmartHome a partir de dos principios: describir lo que realmente puede sostenerse con evidencia e integrar el material visual y documental del equipo sin confundirlo con pruebas de funcionamiento. Por esa razón se distinguen tres capas de lectura: diseño previsto, diseño implementado y elementos históricos de apoyo (prototipos y diagramas conservados en Drive).

Como marco de referencia, la arquitectura se describe por módulos con capas de presentación, aplicación, dominio e infraestructura, según los principios de arquitectura limpia (Martin, 2017) y los patrones de repositorio y capa de servicios (Fowler, 2002). Los atributos de calidad se valoran con el modelo de producto de ISO/IEC 25010 (ISO/IEC, 2023), la autorización por filas usa las políticas de seguridad de PostgreSQL (PostgreSQL Global Development Group, 2025) y la comunicación con los dispositivos sigue el protocolo MQTT 5.0 (OASIS, 2019).

2. Fuentes de diseño consideradas

Tabla 1

Fuentes de diseño consideradas

Fuente	Aporte al diseño	Criterio de uso
Código del backend, web y móvil	Estructura de módulos, rutas, servicios, componentes y contratos.	Fuente principal para el diseño implementado.
Repositorio de base de datos	Modelo lógico/físico, migraciones y versionamiento.	Fuente principal de persistencia.
SRS revisado	Alcance, actores, reglas y requisitos.	Fuente de alineación funcional.
Prototipos y diagramas del equipo	Visualización histórica del sistema.	Se citan como apoyo, no como prueba de producción.

Fuente: elaboración propia.

3. Diseño arquitectónico

3.1 Visión general

SmartHome se entiende mejor como una solución compuesta por cuatro bloques:

1. Cliente web para gestión y consulta desde navegador.
2. Cliente móvil para operación desde Android/Expo.
3. Backend que centraliza reglas de negocio, seguridad, persistencia e integración.
4. Base de datos e integración IoT para almacenar la información y conectar eventos o telemetría.

3.2 Decisiones de arquitectura que sí pueden sostenerse

Tabla 2

Decisiones de arquitectura que sí pueden sostenerse

Decisión	Sustento	Observación
Separación cliente / servidor	Existe frontend web, app móvil y backend independientes.	Es una separación observable en repositorios.
Organización modular del backend	Los módulos del backend agrupan dominios y responsabilidades.	Favorece escalabilidad y mantenibilidad.
Persistencia relacional versionada	PostgreSQL y Liquibase aparecen como piezas centrales.	La fuente de verdad del esquema es el repositorio de BD.
Integración IoT por mensajería	La documentación y el código la relacionan con MQTT y dispositivos compatibles.	Requiere pruebas físicas para su confirmación operativa.
Soporte de tiempo real	Se documenta y se observa la intención de actualizar información en vivo.	Debe validarse en pruebas integradas.

Fuente: elaboración propia.

3.3 Qué no se declara como garantía de diseño implementado

No se presentan como hechos probados, sino como metas o afirmaciones históricas, los siguientes puntos: disponibilidad del 99 %, soporte de 10.000 dispositivos, cumplimiento formal de accesibilidad AA, sincronización offline completa, MFA obligatorio o tiempos de respuesta garantizados en todos los entornos. Estos puntos pueden mantenerse como visión del proyecto, pero no como características verificadas solo con documentación histórica.

4. Diseño por componentes

4.1 Frontend web

El frontend web concentra pantallas de autenticación, panel principal, hogares, dispositivos, consumo, notificaciones y configuración. Su diseño privilegia una navegación lateral o jerárquica y el uso de componentes orientados a administración y consulta de información.

Aporte del material del equipo: el Drive contiene prototipos históricos del dashboard, gestión de dispositivos, configuración de idioma, configuración de notificaciones, reportes de consumo y otros recorridos. Estos prototipos son útiles para ilustrar intención de UX/UI y consistencia visual.

4.2 Frontend móvil

El frontend móvil distribuye las funciones principales en pantallas compactas: inicio, hogares, energía, avisos, perfil y pantallas especializadas. Esta organización responde al uso desde Android y a patrones de navegación táctil.

Aporte del material del equipo: en Drive se hallaron prototipos de inicio de sesión, gestión de dispositivos, reportes, restauración de datos, registro de usuario y otros flujos. En este documento se consideran como referencia de diseño y no como capturas funcionales de la aplicación final.

4.3 Backend

El backend es el núcleo de orquestación del sistema. Su papel es recibir solicitudes desde los clientes, aplicar reglas de negocio, validar acceso, interactuar con la base de datos y coordinar integraciones como notificaciones, telemetría o mensajería. En el diseño académico conviene presentarlo como el punto de convergencia entre autenticación, hogares, dispositivos, consumo y auditoría.

4.4 Capa de datos

La persistencia del proyecto descansa en PostgreSQL y su evolución estructural se gobierna con Liquibase. Por ello, el diseño de base de datos no debe tomarse de diagramas sueltos o documentos antiguos como fuente única, sino contrastarse siempre con el changelog y los scripts vigentes.

5. Patrones y convenciones observables

Tabla 3

Patrones y convenciones observables

Área	Patrón / convención identificable	Valor para el diseño
Backend	Organización modular, controladores, servicios y casos de uso / reglas encapsuladas.	Facilita separación de responsabilidades.
Backend	DTOs, validaciones y control de acceso.	Mantiene contratos claros y protege operaciones

Área	Patrón / convención identificable	Valor para el diseño
		sensibles.
Frontend	Componentización y separación entre vistas, hooks o servicios.	Mejora reutilización y mantenibilidad.
Persistencia	Versionamiento de esquema con Liquibase.	Da trazabilidad a cambios estructurales.
Integración	Uso de APIs y canales de mensajería.	Desacopla clientes del detalle interno del procesamiento.

Fuente: elaboración propia.

6. Diseño de datos: modelo conceptual, lógico y físico

6.1 Modelo conceptual resumido

A nivel conceptual, SmartHome gira alrededor de estas entidades de negocio:

- Usuario
- Rol / membresía
- Hogar
- Zona o espacio
- Dispositivo
- Lectura / consumo / energía
- Notificación / alerta
- Auditoría

6.2 Relación lógica esperada

Tabla 4

Relación lógica esperada

Entidad	Relación principal	Propósito
Usuario	se vincula a uno o varios hogares	compartir acceso y permisos
Hogar	agrupa zonas y dispositivos	contexto operativo del consumo
Dispositivo	pertenece a un hogar o zona	fuente de estado y control
Lectura / consumo	se asocia a dispositivo y/o hogar	análisis energético
Notificación	se dirige al usuario o al hogar	alertas y avisos del sistema
Auditoría	registra acciones relevantes	trazabilidad y control

Fuente: elaboración propia.

6.3 Regla editorial importante

Cuando exista una diferencia entre un diagrama histórico del Drive y el esquema actual del repositorio de base de datos, prevalece el repositorio versionado. Los diagramas históricos sirven para explicar, no para reemplazar la fuente técnica oficial del modelo físico.

7. Diseño de interfaces y prototipos del equipo

La mejora principal solicitada para este documento era incorporar prototipos y diagramas originales del equipo. Aunque esta versión no incrusta imágenes sobre las páginas, sí reorganiza su uso documental para que queden bien clasificados y listos para ser integrados visualmente sin superponer texto.

7.1 Prototipos web identificados

- Dashboard principal
- Gestión de dispositivos
- Gestión de hogares
- Configuración de idioma
- Configuración de notificaciones
- Reportes de consumo energético
- Recomendaciones de ahorro
- Registro, login y recuperación de contraseña

7.2 Prototipos móviles identificados

- Inicio de sesión
- Registro de usuario
- Mis hogares
- Gestión de dispositivos
- Reportes y datos históricos
- Modo offline (como propuesta de diseño)
- Integración con asistentes de voz
- Personalización de tema

7.3 Uso correcto de estos prototipos

1. Deben citarse como prototipos históricos del equipo.
2. No deben presentarse como capturas de una build ejecutada, a menos que exista evidencia adicional.
3. Cada figura debe ir en su propio bloque, con pie de figura y sin superposición de texto.

8. Diagramas del equipo y su papel en la documentación

En el Drive se localizaron diagramas de casos de uso, secuencia, actividad, modelo relacional, despliegue y componentes. Su papel dentro del producto 04 es el siguiente:

Tabla 5

Diagramas del equipo y su papel en la documentación

Tipo de diagrama	Utilidad en el documento
Casos de uso	explicar alcance funcional y actores
Secuencia	explicar interacción entre cliente, backend y persistencia
Actividad	describir decisiones y recorridos operativos
Modelo relacional / ER	apoyar la explicación del diseño de datos
Despliegue	ilustrar la distribución de componentes
Componentes	explicar la división del sistema por módulos o capas

Fuente: elaboración propia.

9. Hallazgos de diseño

1. El diseño del proyecto es más sólido cuando se presenta como arquitectura modular y por componentes, no como un único patrón rígido aplicado a todo.
2. Los prototipos del equipo son un activo valioso para la entrega, siempre que se etiqueten correctamente.
3. El modelo de datos debe documentarse desde Liquibase/PostgreSQL y no desde una imagen antigua aislada.
4. El backend debe describirse como centro de orquestación del sistema, especialmente para autenticación, hogares, dispositivos y consumo.
5. La documentación final debe separar con claridad: diseño propuesto, diseño implementado y funcionalidad pendiente de validación.

10. Conclusión

La presente actualización del producto 04 mejora la coherencia entre la documentación visual del equipo y la evidencia técnica del proyecto. Con esta versión, el diseño del software queda mejor sustentado, más honesto frente a sus límites y más útil para exposición académica, revisión del instructor y continuidad del proyecto.

Referencias

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.

ISO/IEC. (2023). *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model* (ISO/IEC 25010:2023). https://www.iso.org/standard/78176.html

Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.

OASIS. (2019). *MQTT version 5.0* (A. Banks, E. Briggs, K. Borgendale y R. Gupta, Eds.). https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html

PostgreSQL Global Development Group. (2025). *PostgreSQL 18 documentation: Row security policies*. https://www.postgresql.org/docs/18/ddl-rowsecurity.html